

Session 6: Arrays – Detailed Notes

What is an Array?

An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**.

Types of Arrays

1. **1D Array** – Linear list of elements
Example: `int arr[5] = {1, 2, 3, 4, 5};`
 2. **2D Array** – Matrix-like structure
Example:
 3. `int mat[2][3] = {`
 4. `{1, 2, 3},`
 5. `{4, 5, 6}`
 6. `};`
-

Array Initialization

```
int arr[5];           // uninitialized
```

```
int arr[5] = {1,2,3,4,5}; // full init
```

```
int arr[] = {1,2,3};    // size auto
```

2D Initialization

```
int mat[2][2] = {{1,2},{3,4}};
```

Accessing Elements

- Index starts from **0**

`arr[0]` → first element

`arr[n-1]` → last element

Passing Array to Function

```
void printArray(int arr[], int n) {  
    for(int i=0;i<n;i++)  
        cout << arr[i] << " ";  
}
```

 Array size must be passed separately.

Array Problems

Sequential (Linear) Search

Idea:

Check each element one by one.

```
int linearSearch(int arr[], int n, int key){  
    for(int i=0;i<n;i++){  
        if(arr[i] == key)  
            return i;  
    }  
    return -1;  
}
```

 **Time:** $O(n)$

 **Space:** $O(1)$

Binary Search

 Works **only on sorted arrays**

Iterative

```
int binarySearch(int arr[], int n, int key){  
    int low=0, high=n-1;  
    while(low<=high){  
        int mid = low + (high-low)/2;  
        if(arr[mid]==key) return mid;  
        else if(arr[mid]<key) low=mid+1;  
        else high=mid-1;  
    }  
    return -1;  
}
```

Recursive

```
int binarySearchRec(int arr[], int low, int high, int key){
```

```

if(low>high) return -1;

int mid = low + (high-low)/2;

if(arr[mid]==key) return mid;

else if(arr[mid]<key)

    return binarySearchRec(arr, mid+1, high, key);

else

    return binarySearchRec(arr, low, mid-1, key);

}

```

🕒 **Time:** $O(\log n)$

📦 **Space:** $O(\log n)$ (recursion stack)

🔧 Remove Element

Remove all occurrences of a given value.

```

int removeElement(vector<int>& nums, int val) {

    int k = 0;

    for(int i=0;i<nums.size();i++){

        if(nums[i] != val)

            nums[k++] = nums[i];

    }

    return k;

}

```

🕒 $O(n)$ | 📦 $O(1)$

🔧 Remove Duplicates from Sorted Array

```

int removeDuplicates(vector<int>& nums) {

    if(nums.size()==0) return 0;

    int k=1;

    for(int i=1;i<nums.size();i++){

        if(nums[i]!=nums[i-1])

            nums[k++] = nums[i];

    }

}

```

```
    return k;
}
```

🕒 $O(n)$ | 📄 $O(1)$

5 Plus One

Add 1 to a number stored as array of digits.

```
vector<int> plusOne(vector<int>& digits) {
    for(int i=digits.size()-1;i>=0;i--){
        if(digits[i] < 9){
            digits[i]++;
            return digits;
        }
        digits[i] = 0;
    }
    digits.insert(digits.begin(),1);
    return digits;
}
```

🕒 $O(n)$ | 📄 $O(1)$

6 Single Number

Every element appears twice except one.

```
int singleNumber(vector<int>& nums) {
    int ans = 0;
    for(int x: nums)
        ans ^= x;
    return ans;
}
```

💡 XOR trick

🕒 $O(n)$ | 📄 $O(1)$

7 Contains Duplicate

```
bool containsDuplicate(vector<int>& nums) {
    unordered_set<int> s;
    for(int x: nums){
        if(s.count(x)) return true;
        s.insert(x);
    }
    return false;
}
```

🕒 $O(n)$ | 📦 $O(n)$

8 Rotate Array (Right Rotation by k)

```
void rotate(vector<int>& nums, int k) {
    int n = nums.size();
    k %= n;
    reverse(nums.begin(), nums.end());
    reverse(nums.begin(), nums.begin()+k);
    reverse(nums.begin()+k, nums.end());
}
```

🕒 $O(n)$ | 📦 $O(1)$

9 Intersection of Two Arrays

```
vector<int> intersect(vector<int>& nums1, vector<int>& nums2) {
    unordered_map<int,int> mp;
    vector<int> res;

    for(int x: nums1) mp[x]++;
    for(int x: nums2){
        if(mp[x] > 0){
            res.push_back(x);
            mp[x]--;
        }
    }
}
```

```

    }

    return res;
}

🕒  $O(n+m)$  | 📄  $O(n)$ 

```

10 Search in Rotated Sorted Array

```

int search(vector<int>& nums, int target) {
    int l=0, r=nums.size()-1;
    while(l<=r){
        int mid=(l+r)/2;
        if(nums[mid]==target) return mid;

        if(nums[l]<=nums[mid]){
            if(nums[l]<=target && target<nums[mid])
                r=mid-1;
            else l=mid+1;
        } else {
            if(nums[mid]<target && target<=nums[r])
                l=mid+1;
            else r=mid-1;
        }
    }
    return -1;
}

🕒  $O(\log n)$ 

```

11 Search in Rotated Sorted Array II (Duplicates)

Same logic, but:

```

if(nums[l]==nums[mid] && nums[mid]==nums[r]){
    l++; r--;
}

```

Worst case 🕒 $O(n)$

📌 Activity: Find Largest Element in an Array

```
int findLargest(int arr[], int n){  
    int maxVal = arr[0];  
    for(int i=1;i<n;i++){  
        if(arr[i] > maxVal)  
            maxVal = arr[i];  
    }  
    return maxVal;  
}
```

🕒 $O(n)$ | 📄 $O(1)$

🔑 Key Takeaways

- Arrays are **fast but fixed-size**
- Most problems use **two pointers / binary search / hashing**
- Space optimization is often tested
- XOR is powerful for unique element problems